

Spring Boot 포트폴리오 백엔드

학습용 코드 가이드

JPA · Spring Security · JWT · MySQL

프로젝트명	portfolio (com.jaeyoung.portfolio)
대상	Spring Boot · JPA 학습자
빌드 도구	Gradle 9.4 / Java 17
프레임워크	Spring Boot 4.0.6
DB	MySQL 8.x
인증	JWT (jjwt 0.12.6)
문서 작성일	2026-04-28

목차

1. 프로젝트 개요와 학습 포인트
2. 기술 스택 (build.gradle 분석)
3. 계층형 아키텍처 한눈에 보기
4. 진입점: PortfolioApplication
5. domain — JPA 엔티티 모델
6. repository — Spring Data JPA
7. service — 비즈니스 로직
8. controller — REST API 진입점
9. dto — 요청/응답 DTO
10. config — Spring Security와 JWT
11. application.yml — 환경 설정
12. 심화: N+1 문제와 default_batch_fetch_size
13. 학습 체크리스트

1. 프로젝트 개요와 학습 포인트

이 프로젝트는 개인 포트폴리오 사이트의 백엔드 API 서버다. 공개 화면에서는 프로젝트 목록과 상세를 조회하고, 어드민 영역에서는 JWT로 로그인한 관리자가 프로젝트를 CRUD 한다. 규모는 작지만 Spring Boot 기반의 실무에서 가장 자주 쓰는 JPA 연관관계, Spring Security 필터 체인, JWT 인증, 예외 처리, 페이징, 그리고 N+1 문제까지 한 곳에서 학습할 수 있다.

이 자료로 배울 수 있는 것

- 엔티티 간 OneToMany / ManyToOne 매핑과 LAZY 로딩의 동작 원리
- @MappedSuperclass + JPA Auditing으로 created_at/updated_at 자동 채우기
- Spring Data JPA 메서드 네이밍 규칙으로 쿼리 자동 생성
- JWT 발급(JwtTokenProvider)과 검증 필터(OncePerRequestFilter) 구현
- Spring Security의 STATELESS 세션 정책과 인가 규칙 작성
- @Transactional(readOnly = true)와 메서드 단위 쓰기 트랜잭션
- @RestControllerAdvice로 전역 예외를 일관된 JSON 포맷으로 변환
- default_batch_fetch_size로 N+1을 IN-clause 배치 쿼리로 해소

2. 기술 스택 (build.gradle)

핵심 의존성을 한 줄씩 짚으면서 어떤 역할을 하는지 익혀두자.

```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version '4.0.6'  
    id 'io.spring.dependency-management' version '1.1.7'  
}  
  
java { toolchain { languageVersion = JavaLanguageVersion.of(17) } }  
  
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa' // JPA + Hibernate  
    implementation 'org.springframework.boot:spring-boot-starter-security' // 인증/인가  
    implementation 'org.springframework.boot:spring-boot-starter-validation' // @Valid 등  
    implementation 'org.springframework.boot:spring-boot-starter-webmvc' // REST  
    compileOnly 'org.projectlombok:lombok'  
    runtimeOnly 'com.mysql:mysql-connector-j'  
    annotationProcessor 'org.projectlombok:lombok'  
  
    implementation 'io.jsonwebtoken:jjwt-api:0.12.6'  
    runtimeOnly 'io.jsonwebtoken:jjwt-impl:0.12.6'  
    runtimeOnly 'io.jsonwebtoken:jjwt-jackson:0.12.6'  
}
```

스타터 별 역할

- data-jpa: Hibernate, EntityManager, JpaRepository 자동 구성
- security: SecurityFilterChain, BCryptPasswordEncoder 사용
- webmvc: @RestController, DispatcherServlet, Jackson
- jjwt-api/impl/jackson: JWT 생성·검증을 위한 라이브러리(런타임 분리)
- lombok: @Getter, @Builder, @RequiredArgsConstructor로 보일러플레이트 제거

3. 계층형 아키텍처 한눈에 보기

요청은 위에서 아래로 흐르고, 응답은 그 역순으로 돌아온다.

계층	패키지	역할
Controller	controller/	HTTP 요청 수신 · DTO 변환 · 검증 위임
Service	service/	트랜잭션 경계 · 비즈니스 규칙 · 도메인 호출
Repository	repository/	Spring Data JPA로 SQL 자동 생성
Domain	domain/	JPA 엔티티 · 비즈니스 메서드 · 연관관계
DTO	dto/	Layer 간 자료 전달 · Entity 직접 노출 방지
Config	config/	Security, JWT, 초기 데이터 설정

왜 계층을 나누는가? 각 계층이 한 가지 관심사만 책임지게 만들면, Controller 교체(예: REST → GraphQL)나 DB 교체 시 영향 범위가 한 계층으로 국한된다. 또한 Service만 단위 테스트하기 쉬워진다.

4. 진입점: PortfolioApplication

```
@EnableJpaAuditing
@SpringBootApplication
public class PortfolioApplication {
    public static void main(String[] args) {
        SpringApplication.run(PortfolioApplication.class, args);
    }
}
```

애너테이션 풀어보기

- @SpringBootApplication = @Configuration + @EnableAutoConfiguration + @ComponentScan
- 현재 패키지(com.jaeyoung.portfolio)와 그 하위에서 @Component, @Service, @Repository, @Controller 빈을 모두 스캔
- @EnableJpaAuditing: BaseEntity의 @CreatedDate / @LastModifiedDate를 자동으로 채우게 하는 트리거

5. domain — JPA 엔티티 모델

5.1 BaseEntity (공통 시간 컬럼)

```
@Getter
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class BaseEntity {
    @CreatedDate
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;
}
```

- @MappedSuperclass: 자체 테이블은 만들지 않고, 상속한 엔티티에 컬럼만 흡수시키는 베이스 클래스
- @EntityListeners(AuditingEntityListener.class): 영속/수정 시점에 createdAt/updatedAt을 자동 주입
- Application의 @EnableJpaAuditing이 켜져 있어야 동작

5.2 Member · MemberRole

```
@Entity
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
@Table(name = "member")
public class Member extends BaseEntity {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    @Column(nullable = false, length = 100)
    private String password;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private MemberRole role; // ADMIN, USER

    @Builder
    public Member(String username, String password, MemberRole role) { ... }
}
```

- protected 기본 생성자: JPA는 리플렉션으로 인스턴스를 만들 때 기본 생성자가 필요
- 외부 코드의 무분별한 new Member()를 막고 @Builder만 노출
- @Enumerated(EnumType.STRING): enum을 'ADMIN'/'USER' 문자열로 저장 (ORDINAL은 순서 변경에 취약)
- length = 100인 password는 BCrypt 해시(60자)를 안전하게 담기 위함

5.3 Project · ProjectStatus

```
@Entity
public class Project extends BaseEntity {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200) private String title;
    @Column(length = 500) private String summary;

    @Lob @Column(columnDefinition = "LONGTEXT")
    private String content;    // 마크다운 본문

    @Enumerated(EnumType.STRING)
    private ProjectStatus status = ProjectStatus.DRAFT; // DRAFT/PUBLISHED/ARCHIVED

    @Column(name = "view_count", nullable = false)
    private Long viewCount = 0L;

    @OneToMany(mappedBy = "project",
                cascade = CascadeType.ALL,
                orphanRemoval = true)
    private List<ProjectTag> projectTags = new ArrayList<>();

    public void increaseViewCount() { this.viewCount++; }
    public void update(...) { /* 필드 일괄 변경 */ }
    public void clearTags() { this.projectTags.clear(); }
    public void addProjectTag(ProjectTag pt) { this.projectTags.add(pt); }
}
```

- @OneToMany(mappedBy = "project"): 외래키는 ProjectTag 쪽이 가지고, Project는 단지 읽기 매핑
- cascade = ALL + orphanRemoval = true: Project를 저장/삭제할 때 ProjectTag도 함께, 컬렉션에서 빠진 ProjectTag는 자동 DELETE
- 비즈니스 메서드(increaseViewCount, update, clearTags)를 엔티티 안에 두는 것이 풍부한 도메인 모델
- @Lob + LONGTEXT: MySQL에서 큰 텍스트 컬럼을 명시 (마크다운 본문 등)

5.4 Tag · ProjectTag (다대다를 연결 엔티티로 풀기)

```
@Entity
@Table(name = "project_tag",
        uniqueConstraints = @UniqueConstraint(
            name = "uk_project_tag",
            columnNames = {"project_id", "tag_id"}))
public class ProjectTag {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "project_id", nullable = false)
    private Project project;

    @ManyToOne(fetch = FetchType.LAZY)
```



```
@JoinColumn(name = "tag_id", nullable = false)
private Tag tag;
}
```

왜 @ManyToMany를 쓰지 않을까? @ManyToMany는 자동 생성된 조인 테이블에 컬럼을 추가할 수 없고 수정 시 SQL이 비효율적이다. 그래서 실무에서는 거의 항상 연결(중간) 엔티티로 풀어 ProjectTag처럼 직접 매핑한다. 나중에 'tag별 사용 횟수', '태그 추가일' 같은 컬럼이 자연스럽게 붙을 수 있다.

- 두 ManyToOne의 fetch가 모두 LAZY인 이유: 목록 조회 시 불필요한 JOIN을 피해 성능 보호
- uniqueConstraints로 (project_id, tag_id) 중복 매핑을 DB 차원에서 차단

6. repository — Spring Data JPA

인터페이스만 정의하면 Spring Data JPA가 메서드 이름을 파싱해서 SQL을 자동 생성한다. find/exists/count/delete + By + 필드명이 핵심 규칙이다.

6.1 ProjectRepository

```
public interface ProjectRepository extends JpaRepository<Project, Long> {
    Page<Project> findByStatus(ProjectStatus status, Pageable pageable);
    Page<Project> findByTitleContaining(String keyword, Pageable pageable);
    Page<Project> findByStatusAndTitleContaining(
        ProjectStatus status, String keyword, Pageable pageable);
}
```

- JpaRepository<Project, Long> 상속만으로 save/findById/findAll/delete 등 자동 제공
- Page<...> + Pageable: 페이지 번호, 사이즈, 정렬을 한 번에 받고, totalElements/totalPages까지 응답에 포함
- Containing은 SQL의 LIKE %keyword%로 변환됨

6.2 TagRepository · MemberRepository · ProjectTagRepository

```
public interface TagRepository extends JpaRepository<Tag, Long> {
    Optional<Tag> findByName(String name);
    Optional<Tag> findBySlug(String slug);
    boolean existsByName(String name);
}
```

```
public interface MemberRepository extends JpaRepository<Member, Long> {
    Optional<Member> findByUsername(String username);
    boolean existsByUsername(String username);
}
```

```
public interface ProjectTagRepository extends JpaRepository<ProjectTag, Long> {
    List<ProjectTag> findByProjectId(Long projectId);
    void deleteByProjectId(Long projectId);
}
```

Optional vs List vs boolean: 단건은 Optional, 다건은 List, 존재 여부만 알고 싶으면 exists*를 쓰면 SELECT 1 LIMIT 1로 변환되어 가장 가볍다.

7. service — 비즈니스 로직

7.1 AuthService — 로그인과 JWT 발급

```
@Service @RequiredArgsConstructor @Transactional(readOnly = true)
public class AuthService {
    private final MemberRepository memberRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtTokenProvider jwtTokenProvider;

    public LoginResponse login(LoginRequest request) {
        Member member = memberRepository.findByUsername(request.getUsername())
            .orElseThrow(() -> new IllegalArgumentException(
                "아이디 또는 비밀번호가 올바르지 않습니다."));

        if (!passwordEncoder.matches(request.getPassword(), member.getPassword())) {
            throw new IllegalArgumentException(
                "아이디 또는 비밀번호가 올바르지 않습니다.");
        }

        String token = jwtTokenProvider.createToken(member.getUsername());
        return LoginResponse.builder()
            .accessToken(token)
            .username(member.getUsername())
            .build();
    }
}
```

- 아이디 없음 / 비밀번호 불일치 모두 같은 메시지를 던져 사용자 enumeration 공격을 차단
- BCrypt는 단방향이라 평문을 다시 만들 수 없으므로 matches()로 비교
- @RequiredArgsConstructor + final 필드로 생성자 주입 → 테스트 시 Mock 주입이 쉬움

7.2 ProjectService — 트랜잭션과 도메인 호출

```
@Service @RequiredArgsConstructor @Transactional(readOnly = true)
public class ProjectService {

    public Page<ProjectListResponse> getPublishedProjects(Pageable pageable) {
        return projectRepository.findByStatus(ProjectStatus.PUBLISHED, pageable)
            .map(ProjectListResponse::from);
    }

    @Transactional
    public ProjectDetailResponse getProject(Long id) {
        Project project = projectRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("..."));
        project.increaseViewCount(); // ← 변경 감지(dirty checking)
        return ProjectDetailResponse.from(project);
    }

    @Transactional
    public Long createProject(ProjectCreateRequest request) {
        Project project = Project.builder()....build();
    }
}
```

```

        projectRepository.save(project);
        addTagsToProject(project, request.getTagNames());
        return project.getId();
    }
}

```

변경 감지(dirty checking): 영속 상태 엔티티의 필드를 바꾸기만 하면 트랜잭션 커밋 시점에 JPA가 자동으로 UPDATE 문을 만들어 준다. increaseViewCount() 후에 save()를 호출할 필요가 없다는 뜻.

- 클래스 단위 readOnly = true → 조회 메서드는 1차 캐시 외 영속성 컨텍스트의 더티 체크링 비활성화로 약간 가벼워짐
- 쓰기 메서드만 @Transactional을 덧씌워 readOnly = false로 오버라이드
- updateProject에서 projectTagRepository.flush()를 호출하는 이유: 같은 트랜잭션 내에서 DELETE 이후 INSERT 시 (project_id, tag_id) UNIQUE 제약 충돌을 방지하기 위해 DELETE를 즉시 DB에 반영

8. controller — REST API 진입점

8.1 공개용 ProjectController

```
@RestController @RequestMapping("/api/projects")
@RequiredArgsConstructor
public class ProjectController {
    private final ProjectService projectService;

    @GetMapping
    public Page<ProjectListResponse> getProjects(
        @PageableDefault(size = 10, sort = "createdAt",
            direction = Sort.Direction.DESC) Pageable pageable) {
        return projectService.getPublishedProjects(pageable);
    }

    @GetMapping("/{id}")
    public ProjectDetailResponse getProject(@PathVariable Long id) {
        return projectService.getProject(id);
    }
}
```

8.2 어드민용 AdminProjectController (CRUD)

```
@RestController @RequestMapping("/api/admin/projects")
public class AdminProjectController {
    @GetMapping          -> 전체 목록 (DRAFT 포함)
    @GetMapping("/{id}") -> 상세 (조회수 증가 X)
    @PostMapping         -> 생성 → {"id": 5} 반환
    @PutMapping("/{id}") -> 수정 (태그 통째로 갈아끼움)
    @DeleteMapping("/{id}") -> 삭제
}
```

/api/admin/**는 SecurityConfig에서 hasRole("ADMIN")으로 보호된다.

8.3 AuthController · HealthController

```
@PostMapping("/api/auth/login")
public LoginResponse login(@RequestBody LoginRequest request) {
    return authService.login(request);
}

@GetMapping("/api/health")
public Map<String,String> health() {
    return Map.of("status", "UP",
        "message", "Portfolio API is running");
}
```

8.4 GlobalExceptionHandler — 전역 예외 처리

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<Map<String,String>> handleIllegalArgumentException(IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)

```

```

        .body(Map.of("message", e.getMessage()));
    }
    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String,String>> handleException(Exception e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(Map.of("message", "서버 오류가 발생했습니다.));
    }
}

```

- @RestControllerAdvice는 모든 @RestController에 횡단 적용되는 전역 예외 처리기
- Service에서 던진 IllegalArgumentException은 자동으로 400 BAD_REQUEST + JSON으로 변환
- 처리되지 않은 모든 예외는 500으로 떨어지되, 사용자에게는 일반화된 메시지만 노출(스택 트레이스 누출 방지)

9. dto — 요청/응답 DTO

엔티티를 그대로 응답에 노출하면 ① 순환참조, ② 내부 필드 유출, ③ 스펙 변경 시 API 깨짐 같은 문제가 생긴다. 그래서 계층 사이에 DTO(Data Transfer Object)를 두는 것이 표준이다.

응답 DTO 변환 패턴

```
public static ProjectListResponse from(Project project) {
    List<String> tagNames = project.getProjectTags().stream()
        .map(pt -> pt.getTag().getName())
        .toList();

    return ProjectListResponse.builder()
        .id(project.getId())
        .title(project.getTitle())
        .summary(project.getSummary())
        .thumbnailUrl(project.getThumbnailUrl())
        .tags(tagNames)
        .viewCount(project.getViewCount())
        .createdAt(project.getCreatedAt())
        .build();
}
```

위 코드의 `project.getProjectTags()`와 `pt.getTag()`는 모두 LAZY이므로 이 메서드가 트랜잭션 안에서 호출되지 않으면 `LazyInitializationException`이 난다. Service의 메서드가 `readOnly` 트랜잭션 안에서 `.map(...)`을 수행하므로 이 프로젝트는 안전하다.

요청 DTO들

```
@Getter @NoArgsConstructor
public class LoginRequest { String username; String password; }
```

```
@Getter @NoArgsConstructor
public class ProjectCreateRequest {
    String title; String summary; String content;
    String thumbnailUrl; String githubUrl; String demoUrl;
    ProjectStatus status;
    List<String> tagNames; // 태그 이름 배열로 받음
}
```

- @NoArgsConstructor: Jackson이 JSON을 역직렬화할 때 기본 생성자를 호출
- 필드는 final이 아니어야 함(역직렬화 후 setter나 reflection으로 채움)
- tagNames를 List<String>로 받기 때문에 클라이언트는 태그 ID를 몰라도 이름만 보내면 된다

10. config — Spring Security와 JWT

10.1 SecurityConfig — 필터 체인

```
@Configuration @RequiredArgsConstructor
public class SecurityConfig {
    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .cors(c -> c.configurationSource(corsConfigurationSource()))
            .sessionManagement(s ->
                s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.POST, "/api/auth/login").permitAll()
                .requestMatchers(HttpMethod.GET,
                    "/api/projects/**", "/api/tags/**", "/api/health").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtAuthenticationFilter,
                UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

- STATELESS: HttpSession을 만들지 않음 → JWT처럼 토큰 기반 인증과 잘 맞고, 서버 수평 확장이 자유로움
- CSRF는 쿠키 세션 인증에서 의미 있고, 토큰 인증은 보통 비활성화
- JwtAuthenticationFilter를 UsernamePasswordAuthenticationFilter 앞에 끼워 넣어 모든 요청에서 토큰을 먼저 검사
- URL 매칭 순서가 중요: /api/admin/**가 anyRequest()보다 먼저 평가되어 ADMIN 권한 체크가 적용됨

10.2 JwtTokenProvider — 발급과 검증

```
@Component
public class JwtTokenProvider {
    @Value("${jwt.secret}") private String secret;
    @Value("${jwt.expiration}") private long expiration;

    private SecretKey key;

    @PostConstruct
    protected void init() { /* secret HmacSha */ }

    public String createToken(String username) {
        Date now = new Date(); Date exp = new Date(now.getTime() + expiration);
        return Jwts.builder()
```



```

        .subject(username)
        .issuedAt(now)
        .expiration(exp)
        .signWith(key)
        .compact();
    }
    public String getUsername(String token) { ... }
    public boolean validateToken(String token) { try {...}catch(Exception){return false;} }
}

```

10.3 JwtAuthenticationFilter — 매 요청 검증

```

@Component @RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    private final JwtTokenProvider jwtTokenProvider;

    @Override
    protected void doFilterInternal(HttpServletRequest req,
                                    HttpServletResponse res,
                                    FilterChain chain) {
        String token = resolveToken(req); // "Authorization: Bearer xxx" 파싱
        if (token != null && jwtTokenProvider.validateToken(token)) {
            String username = jwtTokenProvider.getUsername(token);
            var auth = new UsernamePasswordAuthenticationToken(
                username, null,
                List.of(new SimpleGrantedAuthority("ROLE_ADMIN")));
            SecurityContextHolder.getContext().setAuthentication(auth);
        }
        chain.doFilter(req, res);
    }
}

```

- OncePerRequestFilter: 한 요청에 한 번만 실행되도록 보장
- Authorization 헤더의 Bearer 접두사를 잘라 토큰만 추출
- 검증 통과 시 SecurityContext에 인증 객체를 심어두고, 이후 SecurityFilterChain의 인가 단계에서 hasRole("ADMIN")이 통과됨
- 이 프로젝트에서는 단일 ADMIN 권한만 다루지만, 실무에서는 토큰의 claim에서 권한 목록을 읽어와 SimpleGrantedAuthority를 동적으로 부여

10.4 AdminInitializer — 첫 부팅 시 관리자 계정 생성

```

@Component @RequiredArgsConstructor
public class AdminInitializer implements CommandLineRunner {
    private final MemberRepository memberRepository;
    private final PasswordEncoder passwordEncoder;
    @Value("${admin.username}") String adminUsername;
    @Value("${admin.password}") String adminPassword;

    @Override
    public void run(String... args) {
        if (memberRepository.existsByUsername(adminUsername)) return;
        Member admin = Member.builder()
            .username(adminUsername)
            .password(passwordEncoder.encode(adminPassword))

```

```
        .role(MemberRole.ADMIN).build();  
        memberRepository.save(admin);  
    }  
}
```

CommandLineRunner는 SpringApplication이 모든 빈을 띄운 직후 1회 실행된다. DB 초기 데이터, 마이그레이션 트리거, 워밍업 등에 활용한다.

11. application.yml — 환경 설정

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/portfolio_db?...
    username: portfolio
    password: ${DB_PASSWORD}    # 외부 환경변수에서 주입
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update          # 운영에서는 none/validate 권장
      show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.MySQLDialect
        default_batch_fetch_size: 100  # ← N+1 방지 핵심
```

```
server:
  port: 8081
```

```
logging:
  level:
    org.hibernate.SQL: debug
    org.hibernate.orm.jdbc.bind: trace
```

```
jwt:
  secret: ${JWT_SECRET:기본키...}
  expiration: 86400000  # 24시간 (ms)
```

```
admin:
  username: ${ADMIN_USERNAME:admin}
  password: ${ADMIN_PASSWORD:admin1234!}
```

- \${VAR:default}: 환경변수가 없으면 default 값을 사용
- ddl-auto: update는 학습용. 실서비스는 Flyway/Liquibase 마이그레이션이 정석
- show-sql / format_sql / bind:trace의 조합으로 실제 실행 SQL과 파라미터까지 로그에서 확인 가능
- default_batch_fetch_size: 100은 LAZY 컬렉션-프록시를 IN-clause 100개씩 묶어서 가져옴 (다음 장 참조)

12. 심화: N+1 문제와 default_batch_fetch_size

12.1 N+1이란?

부모 N개를 조회한 뒤(쿼리 1번), 각 부모의 LAZY 자식 컬렉션을 만질 때마다 추가로 N번의 SELECT가 더 나가는 상황이다. 한 요청에서 쿼리가 1 + N (또는 1 + N + N×M)으로 폭증해 DB 라운드트립이 늘고 응답 시간이 비례해 길어진다.

12.2 이 프로젝트의 사례

```
// ProjectListResponse.from(project)
List<String> tagNames = project.getProjectTags().stream() // ← LAZY 트리거
    .map(pt -> pt.getTag().getName()) // ← 여기서 또 LAZY
    .toList();
```

페이징 사이즈 20, 평균 태그 3개라면:

- 1번: SELECT ... FROM project WHERE status = ?
- 20번: 각 project의 projectTags 컬렉션 → SELECT ... FROM project_tag WHERE project_id = ?
- 60번: 각 ProjectTag의 tag 프록시 → SELECT ... FROM tag WHERE id = ?
- 합계 약 81번 ← 페이지 한 번 부르는데 이렇게 많이 나간다

12.3 해결: default_batch_fetch_size = 100

Hibernate에 "LAZY 로딩이 필요할 때 그 시점에 모인 ID들을 한 번에 IN-clause로 묶어서 보내라"고 알려주는 옵션이다. 코드를 한 줄도 바꾸지 않고 application.yml 한 줄로 끝난다.

```
spring:
  jpa:
    properties:
      hibernate:
        default_batch_fetch_size: 100
```

적용 후 실제 SQL 로그

```
Hibernate: select ... from project where status=? ...
Hibernate: select ... from project_tag pt where pt.project_id in (?, ?, ?, ... 100 )
Hibernate: select ... from tag t where t.id in (?, ?, ?, ... 100 )
```

81번 → 3번으로 감소. 페이지 사이즈를 더 키워도 쿼리 수는 거의 그대로 유지된다 (IN 안의 파라미터만 늘어남).

12.4 다른 방법과의 비교

방법	장점	단점/주의
fetch join (JPQL)	한 방에 모두 가져옴	컬렉션을 fetch하면 페이징이 메모리에서 일어남(경고 발생)
@EntityGraph	선언적이고 깔끔	fetch join과 동일한 컬렉션 페이징 한계
@BatchSize	필드/엔티티 단위 세밀 제어	엔티티마다 붙여야 해서 일관성 관리가 번거로움
default_batch_fetch_size (전역)	application.yml 한 줄, 코드 무수정, 페이징과 호환 쿼리 수가 1이 아니라 1 + (count/size)로 약간 늘어남(보통)	

13. 학습 체크리스트

아래 질문에 막힘없이 답할 수 있다면 이 프로젝트는 충분히 학습한 것이다.

- Q1. @MappedSuperclass를 쓰면 테이블이 어떻게 만들어지는가?
- Q2. JPA의 변경 감지(dirty checking)는 언제 SQL을 만드는가?
- Q3. @OneToMany(mappedBy = ...)에서 mappedBy가 가리키는 것은 무엇인가?
- Q4. ManyToMany를 직접 쓰지 않고 연결 엔티티로 푸는 이유 3가지를 말할 수 있는가?
- Q5. BCryptPasswordEncoder.matches는 왜 평문 복호화 없이 비교가 가능한가?
- Q6. Spring Security 필터 체인에서 JwtAuthenticationFilter가 들어가는 위치는?
- Q7. STATELESS 세션 정책을 쓰면 무엇이 달라지는가?
- Q8. @RestControllerAdvice의 적용 범위는?
- Q9. Page와 Pageable이 자동으로 만들어 주는 SQL은 어떤 형태인가?
- Q10. default_batch_fetch_size를 키우는 것의 트레이드오프는?
- Q11. DTO를 두지 않으면 어떤 문제가 생길 수 있는가?
- Q12. @Transactional(readOnly = true)를 클래스에 걸면 메서드 단위 @Transactional은 어떻게 동작하나?

다음 단계 추천: ① QueryDSL 도입으로 동적 검색 쿼리 작성, ② Refresh Token 추가, ③ Soft delete(@SQLDelete), ④ 통합 테스트(@SpringBootTest + Testcontainers), ⑤ Caffeine/Redis로 인기 프로젝트 캐싱.